Studio Nx

Backend(FastAPI/Python) 07 — get_current_user_optional()

2026-10-12

Backend(FastAPI/Python) 07 — `get_current_user_optional()`

항목	내용
파일	`studionx/backend/app/api/deps.py`
작업	P1-03 백엔드 공통: 로그인이 선택인 API (PRD 01 FR-112)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	06 `get_current_user()` · 08 `User` 모델

1. 하는 일

로그인한 회원이면 그 회원을, 아니면 `None`(손님)을 라우터에 넘깁니다. **401 오류를 내지 않습니다.** 검사 내용은 06번과 같고, 실패했을 때의 처리만 다릅니다.

	06 `get_current_user()`	07 이 함수
로그인한 `ACTIVE` 회원	회원을 넘깁니다	회원을 넘깁니다
헤더 없음, 토큰 틀림, 회원 없음, 정지, 탈퇴	401 오류를 내고 라우터를 실행하지 않습니다	`None`을 넘기고, 라우터는 손님으로 실행됩니다
라우터에 적는 이름	`user: CurrentUser`	`user: OptionalUser`

2. 언제 쓰나

지금 쓰는 곳은 **문의 등록** `POST /api/v1/inquiries` 하나입니다 (home.html 아래쪽의 Contact 폼).

보내는 사람	이 함수의 결과	저장되는 문의 (`inquiries` 테이블)
로그인하지 않은 방문자	`None`	`user_id = NULL`이고, 폼에 적은 이메일로만 답장합니다
로그인한 7번 회원	7번 회원	`user_id = 7`이므로, 관리자가 어느 회원의 문의인지 알 수 있습니다
토큰이 만료된 회원	`None`	`user_id = NULL`이고, 문의는 그대로 접수됩니다

```
def create_inquiry(data: InquiryCreateRequest, request: Request, user: OptionalUser, db: Db):
    ...   # P1-12: user 가 있으면 inquiry.user_id = user.user_id
```

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import decode_access_token
```

```
from app.db.session import get_db
from app.models import User
```

`deps.py` 맨 위의 import입니다 (06번과 같음). 이 함수는 **`AppError`**만 직접 씁니다. 나머지 검사는 06번 `get_current_user()`를 불러서 합니다.

함수

```
def get_current_user_optional(
    credentials: Bearer,
    db: Db,
) -> User | None:
    if credentials is None:
        return None
    try:
        return get_current_user(credentials, db)
    except AppError:
        return None


OptionalUser = Annotated[User | None, Depends(get_current_user_optional)]
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다

식	뜻
`-> User | None`	회원 또는 `None`을 돌려준다는 표시입니다
`get_current_user(credentials, db)`	06번을 **보통 함수처럼 직접** 부릅니다. `Depends`를 거치지 않으므로, 이미 받은 `credentials`와 `db`를 그대로 넘깁니다
`try: … except AppError: return None`	06번이 던진 401(`AppError`)을 잡아서 `None`으로 바꿉니다. 05번의 토큰 오류도 `AppError`라서 함께 잡힙니다
`except AppError` (모든 오류를 잡지 않음)	DB 연결이 끊긴 오류 같은 것은 잡지 않습니다. 그런 오류는 04번이 500으로 처리합니다

4. 흐름

1. 헤더가 없으면 `None`을 돌려줍니다. DB에 묻지 않습니다.
2. 헤더가 있으면 06번과 같은 검사를 합니다.
3. 통과하면 회원을, 401이 나면 `None`을 돌려줍니다.

5. 왜 이렇게

선택	이유
실패해도 401을 내지 않음	문의는 손님도 보낼 수 있습니다. 만료된 토큰 때문에 문의가 거절되면 안 됩니다
검사를 06번에 맡김	검사 규칙(토큰, 회원 없음, 정지, 탈퇴)이 한 곳에만 있게 됩니다. 06번을 고치면 07번도 같이 바뀝니다

선택	이유
`AppError`만 잡음	DB 오류까지 `None`으로 바꾸면 서버 고장이 숨겨집니다. 고장은 04번이 500으로 알리고 Sentry에 기록합니다
정지되거나 탈퇴한 회원도 `None`	그 회원 이름으로 연결하지 않고 손님 문의로 받습니다

P1-12에서 알아 둘 점: Access Token은 메모리에만 있어서 home.html을 새로 열면 비어 있습니다 (JS 01). 로그인한 회원의 문의를 회원과 연결하려면, `home.js`가 보내기 전에 `isLoggedIn()`(JS 06)으로 토큰을 먼저 받아 두어야 합니다.

6. 시험 (`tests/test_deps.py`에 6개를 추가해서 모두 13개)

06번과 같은 가짜 세션(`FakeDb`)을 쓰고, 시험용 앱에 `GET /maybe(user: OptionalUser` → `{"user_id": 7 또는 null}`)를 만들었습니다.

경우	결과
정상 토큰과 `ACTIVE` 회원 7	200 `{"user_id": 7}`
헤더 없음	200 `{"user_id": null}`이고, DB에 묻지 않습니다
틀린 토큰	200 `{"user_id": null}`
토큰의 회원 99가 DB에 없음	200 `{"user_id": null}`
`DELETED` 회원	200 `{"user_id": null}`
`Authorization: Basic …`	200 `{"user_id": null}`

전체 시험 48개가 통과했고, `ruff` 검사도 통과했습니다.